| Life Polaroid |

PM -> curs 5

Preemptive Concurrency -> Parallelism

- MCU -> sunt de obicei SINGLE CORE
- Task-urile în paralel necesita un OS, și pot fi puse pe (pauză) oricând
- Switching-ul între sarcini este costisitor: salvezi contextul (registre, memorie) și îl restaurezi pt alt task
- Task-urile care fac foarte mult I/O -> asta face ca timpul de comutare să dureze mai mult ca TIMPUL DE PROCESARE
  (exp aștepți date de la un senzor) (switching time)

Cooperative Concurrency

- task-urile nu pot fi întrerupte
- componentele hardware funcționează în mod asincron (livrează datele când sunt gata, nu când le ceri)
- task-urile cooperează astfel: un task cedează procesorul când știe că nu mai are ce face (exp așteaptă date)
- nu e nevoie de un OS, totul merge într-un sg flux CONTROLAT MANUAL
- Nu (!) penalizări pentru salvarea/restaurarea stării pt că task-ul știe singur când să "iasă" și "să lase loc" altora.

RP2350 -> este un DUAL CORE MCU, folosim doar un core

! întreruperile hardware (ISR) pot prelua controlul oricând chiar și în COOPERATIVE CONCURRENCY

Asynchronous Executor în Embassy

- executorul gestionează task-urile asincrone
- task-urile pot ceda controlul folosind .await când trebuie să aștepte un eveniment

#[embassy_executor::main]
- marchează funcția principală
- inițializează și pornește scheduler-ul Embassy

Parametrul pool_size -> definește câte copii ale task-ului pot fi active simultan

În task-uri se folosește (.await) ca să eliberezi procesorul cât timp se așteaptă un eveniment (altfel se blochează tot MCU-ul)

Embassy suportă task-uri cu PRIORITĂȚI DIFERITE

- un IRQ (Interrupt Request) de prioritate mare poate lăsa task-uri urgente
- SWI (Software Interrupts) pot lansa task-uri la alte niveluri de priorități
- Task-urile cu priorități mai mici sunt întrerupte dacă vine un task cu prioritate mai mare

Ce este Future în Rust?

Future -> valoare disponibilă în viitor
- echivalentul este PROMISE în java.

Are două stări:
- PENDING -> încă nu e gata
- Ready(T) -> a fost calculat și putem folosi rezultatul T

Un Future are o funcție cheie

fn poll(&mut self) -> Poll<Self::Output>

poll -> executor
-> dacă task-ul este gata, întoarce Poll::Ready(valoare)
-> dacă task-ul nu e gata, întoarce Poll::Pending

Async + Rust
↳ async fn ceva() {}
↳ rust rescrie funcția ta într-un obiect FUTURE
↳ RUST nu rulează singur Future-ul
↳ e nevoie de un executor care să cheme POLL pt FUTURE

CONTEXT și WAKER
↳ poll primește un CONTEXT care conține un WAKER
↳ WAKER-ul e folosit de un task să îi spună executorului „Hei sunt gata acum, și poll-ează-mă din nou"

Embassy + Future
↳ ① Task-urile se înregistrează la executor
↳ ② Când așteaptă un ex. (.await) intră în sleep
↳ un ISR sau un eveniment hardware TREZEȘTE TASK-UL folosind WAKER
↳ executorul rulează task-ul trezit

ACCESUL

① Accesul semultan:
↳ RUST-ul interzice scrierea semultană în același resursă
↳ dacă două task-uri doresc să modifice aceeași resursă trebuie SERIALIZATE

② Accesul EXCLUSIV
↳ se folosește un mecanism de Mutual Exclusion (Mutex)
↳ MUTEXUL se asigură că: -> doar un sg task are voie să scrie la un moment dat
-> celelalte task-uri trb să aștepte

Tipuri de mutex
↳ NOOP MUTEX -> dotă între task-uri cu același executor
↳ CRITICAL SELECTION MUTEX -> dotă între mai multe executoare, ISR, și core-uri
↳ THREAD MODE MUTEX -> dotă între task-uri normale, fără ISR, pe un singur CORE

BLOCKING vs Async Mutex

↳ putem folosi (.await) chiar și când avem lock-ul, MUTEXUL va elibera lock-ul automat cât timp task-ul așteaptă)

↳ Nu putem folosi .await (pt că se blochează complet)

- BLOCKING: SHARED_DATA.lock(|f| {...});
- Async: let mut data = SHARED_DATA.lock().await

Channel-uri

↳ folosite pentru a trimite date între task-uri

① Channel → Multiple Producer Multiple Consumer (MPMC)
② Priority Channel → MPMC, dar cu priorități. Cele urgente trec primele
③ Signal → semnalizează un eveniment, doar ultimul semnal contează, unui sg consumator
④ PubSubChannel → Fiecare consumator primește toate mesajele (broadcast)

CURS 6

ISR (Interrupt Service Routine) = funcție specială APELATĂ automat de când un anumit eveniment hardware are loc
Evenimentele pot fi: → apăsarea unui buton, sosirea unui pachet UART
Când un ISR se întâmplă → procesorul întrerupe ce făcea
↳ sare la ISR-ul care făcea acel eveniment
↳ după ce ISR-ul termină, procesorul reia EXACT unde a rămas

Cum afectează ISR-ul concurența?

① ISR-urile → introduc preemțiune ⇒ un task normal poate fi întrerupt în orice moment dacă un INTERRUPT

② Dacă ISR-ul modifică date comune cu alte task-uri
↳ trebuie protejate cu mecanisme de sincronizare (mutex, secțiuni critice) → altfel apar pb de race condition

DMA → hardware specializat în microcontrolere care face transferuri de date între memorie și periferice fără implicarea procesorului

Plusuri: → scutește MCU de operații tip MEMORY-TO-MEMORY → CPU-ul nu mai trebuie să facă manual (mov, load, store) între adrese
→ MMIO (Memory Mapped I/O) este implicat → transferuri între memorie și periferice (UART, SPI, ADC)
→ când un transfer se termină, DMA ridică un interrupt ca să anunțe CPU-ul că e gata.

! DMA nu știe conținutul cache-ului procesorului
Dacă ai cache activat pe memorie: → zona trb marcată ca "nocache" (dacă MCU permite)
→ fie se face flush manual la cache și după transferul DMA

DMA → eliberează procesorul pt alte calcule, crescând performanța
↳ atenție la sincronizarea MEMORIEI CACHE și FIZICE, pt a nu se transfera date vechi sau incomplete.

UART

↳ metodă comunicare serială între două dispozitive
↳ folosește două fire independente : ① TX → transmitere
② RX → receptie

Firele sunt încrucișate între dispozitive (TX-ul unuia merge la RX-ul celuilalt)

Proprietățile unui dispozitiv UART
① BITS → câți biți sunt în payload (5-9 biți)
② PARITY → se adaugă sau nu un bit de paritate
③ STOP → 1 sau 2 biți de stop
④ BAUD RATE → câți biți sunt trimiși pe secundă (9600, 115200)

Formulă Baud Rate

$$\text{baud Rate} = \frac{f_{clock}}{\text{divider} \times (1 + \text{payload}_{bits} + \text{parity}_{bits} + \text{stop}_{bits})}$$

Tipuri UART
① TTL UART → folosit la comunicații scurte 3.3V sau 5V
② RS232 → pt cabluri lungi între -12V și 12V
③ RS485 → industrial, diferențial (două fire pt diferența de tensiune)

① Receiver (RX) → folosește un Shift Register pt a citi fiecare bit serial
→ poate avea FIFO (coadă pt datele recepționate)
→ generează întreruperi (interrupt) când : → se primește un caracter
→ FIFO-ul este plin sau jumătate plin
② Transmitter (TX) → folosește un Shift Register pt a trimite fiecare bit serial
→ generează întreruperi când : ① se trimite un caracter
② FIFO-ul este gol sau jumătate gol

! ① Duplex complet → se poate transmite/primi simultan
② Nu (!) semnal de ceas transmis între dispozitive → receptorul trebuie să își sincronizeze ceasul intern
③ Pe RP2350 (Raspberry Pi Pico 2 W) → (!) două UART-uri hardware

UART cu Embassy RP

① Synchronous API : `uart.blocking_write(...)`
`uart.blocking_read(...)`

② Asynchronous API (cu DMA și întreruperi) : `uart.write(..).await`

UART = protocolul de bază pt comunicația serială între două dispozitive
Embassy oferă variante sincrone + asincrone pt a lucra eficient cu UART-ul pe microcontrolere

SPI

↳ folosit pentru comunicarea între circuite integrate (microcontroler și senzori)

Avem două tipuri de senzori
① main (master) → controlează comunicarea (MCU-ul)
② sub (slave) → răspunde comenzilor master-ului (de obicei un senzor)

Firele folosite în SPI
① MOSI → Master OUT, Slave IN → date de la master la slave
② MISO → Master IN, SLAVE OUT → date de la slave la master
③ CLK → Clock → ceas generat de master, sincronizează scrierea/citirea
④ CS → Chip Select → activarea individuală a unui slave (nu e standard SPI, dar e esențial în practică)

Cum funcționează o transmisie SPI

① Master setează (CS) pe (LOW) pt după dorit
② Master pune primul bit pe MOSI
③ Slave pune primul bit pe MISO
④ Master pornește CLK (ceasul)
⑤ La rising edge sau falling edge (depinde de mod), datele sunt citite/scrise

CPOL și CPHA definesc când anume sunt copiate și transmise datele în raport cu semnalul de ceas (CLK)

MODE	CPOL	CPHA	SEMNIFICAȚIE
0	0	0	Rising Edge, la înc ciclului
1	0	1	Rising Edge, la mijlocul ciclului
2	1	0	Falling edge, la înc ciclului
3	1	1	Falling edge, la mijlocul ciclului

- Daisy Chaining -> = conectează mai multe slave-uri în lanț: ① Masterul trimite date către primul slave
② Fiecare slave trimite mai departe datele către urm
↳ = utilizează mai ales LED-uri adresabile sau unele ADC-uri

- SPI → protocol serial, rapid, pentru conectarea mai multor disp periferice la un controler

- Embassy oferă atât metode sincrone și asincrone (cu suport DMA) pt a face transmisii eficiente).

! Imp SPI
① Transmisie → duplex complet
② Clock → sursă unică de ceas de la master
③ Nr. dispozitive → 1 master + mai mulți slave
④ Viteză → nu (!) limită teoretică, limitată practic de hardware

SENZORI

- Tipuri de senzori:
  ① analog senzor: doar traductor (un senzor de temperatură simplu), necesită un ADC pentru conversie digital
  ② digital senzor: traductor + ADC + procesor MCU integrat, necesită ieșire digitală pe SPI / I2C

ANALOG = semnal de tensiune 2gomotos → NECESITĂ PROCESARE
DIGITAL = semnal digital gata de folosit → prin magistrala SPI sau I2C

Exp senzor digital: BMP280
↳ senzor digital fabricat de BOSCH
Are: → BUS SPI sau I2C
→ Registru de control pt config
→ Registru de date pt citirea presiunii

CITIRE DATE dintr-un senzor digital prin SPI
① pui linia CHIP SELECT (CS) pe LOW (activezi senzorul)
② trimiti adresa registrului dorit + bitul de citire (1<<7)
③ primești răsp prin SPI
④ Ridici CS înapoi pe HIGH

! ① Senzorii analogici = necesită ADC și procesare 2gomot
② Senzori digitali = comunicare magistrală SPI sau I2C
③ Citire SPI = trimitem adresă cu bit citire setat
④ Scriere SPI = trimitem adresă cu bit de scriere
⑤ Embossy = permite atât operații sincrone cât și asincrone cu SPI

I2C

↳ folosit pt comunicarea între circuite integrate

Are două roluri

① CONTROLLER (master) → inițiază comunicarea (exp: microcontrolerul)

② TARGET (slave) → răspunde comenzilor master-ului (exp senzor)

Firele și adresele

① SDA : Serial Data Line → pt date bidirecționale

② SCL : Serial Clock Line → semnal de ceas generat de controller

Fiecare dispozitiv target are o adresă unică (7 sau 10 biți)

Liniile sunt PULL-UP (HIGH) și sunt trase în jos (LOW) când se scrie

Cum funcționează o transmisie I2C → 7 bit address

① controllerul trimite semnalul START

② controllerul trimite adresa dispozitivului TARGET + bit de citire/scriere (R/W)

③ Target-ul răspunde cu ACK sau NACK

④ Controllerul sau targetul transmite datele

⑤ Controllerul transmite semnalul STOP

10-bit address

↳ similar dar se trimit două coduri pt adresa completă de 10 biți

! Transmisie = half-duplex (cum sg sens la un moment dat)

Ceas = sincronizat de controller

Fire = SDA/SCL (aceleași fir de culine/servere) și un fir pt ceas

Dispozitive = un controller, mai multe target-uri (un receiver, un transmitter)

I2C → protocol serial, a fire master-slave

SDA/SCL → linie date și linie ceas

Adrese → 7 sau 10 biți

Moduri → viteză până la 5 Mbit/s (Ultra-fast)

Embossy → suportă atât operații sincron cât și asincron pt I2C

Folosit la: → senzori

→ display-uri mici

→ RP2350 are două dispozitive I2C

USB (universal Serial BUS)

- folosit pentru comunicatie intre un host (calculator) și mai multe device-uri (mouse, tastatură, stick USB)
- un dispozitiv primește o adresa 7-bit la conectare
- putem conecta până la 127 de dispozitive la un host, folosind HUBS.
- structura este IERARHICĂ: HOST -> HUB -> DEVICE-URI

Arhitectura de comunicare

1. HOST -> inițiază comunicarea
2. DEVICE -> răspunde la cererile hostului
3. ENDPOINTS -> buffere pt date IN and OUT
4. BUS -> 2 fire de date (DP/DM) folosind semnal diferential
5. CLOCK -> 48 MHz intern (MCU -> sincronizeaza local)

Tipuri pachete USB

1. TOKEN -> inițiază transferul (OUT, IN, SETUP)
2. DATA -> transportă date reale (DATA0, DATA 1)
3. HANDSHAKE -> confirmă transferul (ACK, NACK, STALL)

Moduri de transfer

1. control = configurare dispozitiv, folosit pt INITIALIZARE, SETARE, DESCRIPTOR
2. ISOCHRONOUS = viteză mare, fără garanția integrității, folosit pt audio/video streaming
3. BULK = fiabilitate, viteză mai mică, folosit pt transfer de fișiere mari
4. Interrupt = latență garantată, folosit pt mouse, tastaturi

Descriptori USB

- device descriptor -> inf generale despre dispozitiv
- configuration descriptor -> configurații posibile
- interface descriptor -> funcții / servicii oferite de dispozitiv
- endpoint descriptor -> cum sunt transferate datele

Viteze USB

- low speed -> 1.5 Mbit/s
- full speed -> 12 Mbit/s
- high speed -> 480 Mbit/s

Embassy + USB API pe RP2350

- config dispozitivului: `embassy_usb::Builder`
- crearea endpointurilor: `bulk_in`, `bulk_out`

API-uri pt host (PC)
- folosind crate-ul `NUSB`

① USB Transfer = TOKEN → Data → Handshake
② dispozitive = host utilizează, device răspunde
③ descriptori - DEVICE, CONFIGURATION, INTERFACE, ENDPOINT
④ embassy USB = putem configura device-uri USB embedded în RUST
⑤ Moduri de transfer = CONTROL, BULK, INTERRUPT, ISOCHRONOUS

Senzori I2C

- Tipuri de senzori
  - ① analog → doar traductor analogic (exp variază tensiunea) → necesită ADC și filtrare zgomot
  - ② digital → traductor + ADC + microcontroler intern. Livrează date direct digital, prin magistrală (I2C/SPI)

Exp senzor digital: BMP280 (senzor presiune de la Bosch)
- ↳ comunică prin I2C sau SPI
- ↳ INTERN: traductor presiune → ADC → MCU intern → output digital

Harta regiștrilor BMP280
- ↳ regiștri importanți: ① ctrl_meas (0xF4) - configurezi modul de măsurare
  ② press_lsb (0xF8) - citești valoarea presiunii

write_read = citim date dintr-un registru
write = scriem date într-un registru

LAB 2 -> GPIO

↳ **General-Purpose Input/Output** -> este o parte esențială a sistemelor embedded -> este esențial în legătura dintre MICROCONTROLERE și MICROPROCESOARE cu lumea exterioară

Pinii unui microcontroller sau a unui microprocesor pot fi configurați ca intrări sau ieșiri
(Permit comunicarea cu componentele externe (LED-uri, butoane, senzori, etc)

Protocoalele standard de comunicare sunt SPI, I2C, PCM, PWM, iar comunicarea serială e facută cu ajutorul pinilor GPIO.
Sunt două tipuri de pini: digitali și analog.

Configurarea pinilor GPIO

↳ pot să fie folosiți ca: LED-uri, motoare, etc

Un pin poate fi configurat fie ca "INTRARE", fie ca "IEȘIRE" -> dar nu ambele în același timp

Cod aprindere led:

anunțăm programul la LED-ul e conectat la PINUL numit LED

① `let mut led = pins.led.into_push_pull_output();`

transf pinul în ieșire (output) ai să putem aprinde sau stinge ledul

`led.set_high();`
`led.set_low();`
↳ trimitem 0 (oprim curentul) = oprim LED-ul
↳ trimitem 1 (curent) = aprindem LED-ul

la pinul is e conectat butonul

->configurăm ca intrare și activăm „pull up resistor" (fără apăsare = pinul e automat 1/High)

② `let button = pins.gpio1s.into_pull_up_input();`

```
if button.is_low().unwrap(){
    // butonul a fost apăsat
}
```

↳ verif dacă semnalul e jos

-> dacă butonul e apăsat, circuitul se închide => semnalul devine 0 => codul din {} se execută

(!) fenomenul de DEBOUNCE la butoane -> când apăsăm un buton din cauza vibrațiilor mecanice, semnalul nu trece frumos de la 1 la 0
Altfel se produc erori
Soluție: așteptăm câteva milisecunde

`Timer::after_millis(20).await // asteapta 20 ms`

Folosirea ASYNC/AWAIT pentru multitasking
Fără să scriem loop, putem scrie mai multe task-uri care merg în paralel
Cu `#[embassy_executor::task]` și await -> pt a aștepta fără să blocăm procesul

```
#[embassy_executor::main]
async fn main(_spawner: Spawner){
    let p = embassy_rp::init(Default::default()); -> inițializează placuța
    let mut led = p.PIN_25.into_push_pull_output();
    loop {
        led.set_high(); // aprind ledul
        Time::after_millis(500).await, // astept 5s
        led.set_low();          // sting ledul
        Time::after_millis(500).await;    // astept 5s
    }
}
```

Spawn → lansează două sarcini care merg în paralel

Cum funcționează GPIO în RP2040/RP2350?

① Pads → controlează pinul fizic: curent maxim, rezistențe de pull-up/down
② IO Bank0 → face legătura între perifericele interne și pini

GPIO poate fi folosit de:
① SIO (funcție de GPIO) → controlează pinii interni, și e cel mai des folosit de programatori
② SPI_RX
③ I2C0_SDA
④ UART0_TX

Multiplexare - fiecare PIN poate avea mai multe funcții - IO Bank0 decide ce rol are pinul în acel moment

① embassy-executor - execută tasksuri async
② embassy-time - oferă delay-uri și timere
③ embassy-usb, embassy-net → comunicare USB/Network
④ defmt → logging eficient pt embedded
⑤ cortex-m-rt, cortex-m → runtime pt ARM, cortex-M
⑥ rp_pac → acces direct la regiștrii (PAC)

LAB 3

① Semnale: ① digitale: 0 sau 1 (două nivele)
② analogice: valori continue (sunet, lumină)

② PWM → simulează un semnal analogic folosind doar 0 și 1
→ schimbăm „cât de mult timp" semnalul stă sus (high)

③ ADC → transformă un semnal analogic (tensiune) în date digitale

④ Ceas → circuit care oscilează constant între HIGH și LOW
→ cel mai precis ceas: oscilator cu cristal (XOSC)

⑤ Contoare → țin evidența nr de impulsuri pe ceas
→ când ajung la maxim, revin la 0 (overflow)
→ 3 regiștri importanți: value → valoare curentă
direction → UP sau DOWN
reset → când se resetează

Un counter de 8 biți numără de la 0 la 255, apoi revine la 0

① SYST_CVR → valoarea actuală a timerului
② SYST_RVR → valoarea la care se resetează
③ SYST_CSR_SET → control (pornire/oprire, întreruperi)

⚠ RP2350 → are două periferice timer (fiecare de câte 4 la RP2040)
→ timerele sunt MONOTONICE (nu dau overflow)

PWM = Pulse Width Modulation

let pwm = &mut p.PWM_CH6
<-> PWM pe canalul 6

set_divider(1) = nu scadem frecvent - folosim frecventa completa (125 MHz)
set_top(100) = PWM numara de la 0 la 100, deci avem o rezolutie de 1%

set_compare(...) : cat din TOP e semnal HIGH

led.set_function(PinFunction::PWM) = ii spunem pinului sa foloseasca functia PWM, nu GPIO normal

ADC -> Analog to Digital Converter = se citeste un semnal analogic si se transforma in val digitala.

let adc = &mut p.ADC
adc.enable = pornim perifericul ADC
let mut pin = p.PIN_28 => pinul conectat la potentiometru

pin.set_function(PinFunction::Null) -> pinul nu mai e GPIO digital -> devine analogic

adc.read(&mut pin) -> citim o valoare de la 0 la 4095 (pe 12 biti)

PWM -> controleaza semnale digitale pt a simula valori analogice (exp: intensitate LED)
ADC -> citeste tensiuni analogice si le transforma in numere (ex unui potentiometru)

LAB 4

Ce e un async fn? -> o functie care „pune pauza" si spune „Astept ceva, dar intre timp fa altceva"
Task-ul asincron e pornit de: spawner.spawn(...)

button.wait_for_falling_edge() -> butonul e apasat

join! -> asteapta tot

```
join!(
    task_led(),
    task_buton(),
    task_uart()
);
```

① async fn -> functie care poate fi intrerupta
② .await -> asteapta fara sa blocheze tot programul
③ Future -> o promisiune ca se va intampla ceva in viitor
④ Executor -> manager care coordoneaza ce task ruleaza cand
⑤ spawn() -> porneste un task nou
⑥ select! -> alege primul FUTURE! care termina
⑦ join! -> asteapta toate Future-urile

LAB 5

SPI → protocol de comunicare sincron, rapid, între
① un dispozitiv principal (main / master)
② unul sau mai multe sub-dispozitive (sub / slave)

Linii utilizate

① CLK → clock-generat de master
② MOSI → Master Out Slave In → date de la master la slave
③ MISO → date de la slave la master
④ CS → Chip Select → activează un slave, fiecare slave are propriul CS

Protocolul este FULL-DUPLEX → se transmit datele în ambele direcții simultan

Slot LOW → se activează

Exp transmisie SPI
① Masterul setează CS pe LOW → activează slave-ul
② Master scrie un bit pe MOSI, iar slave-ul scrie un bit pe MISO
③ Master pornește ceasul CLK
④ La fiecare "rising edge" sau "falling edge" → se face citirea / scrierea semnalului
⑤ Se repetă până la final

DMA → Direct Memory Acces
① DMA → mută date fără implicarea procesorului
② Poate transmite / cati date SPI fără să blocheze firul principal
③ Când termină trimite un INTERRUPT

LAB 6

I2C → Inter-Integrated Circuit
↳ protocol bidirectional și folosește doar două fire
① SDA (date)
② SCL (ceas)

Un dispozitiv CONTROLLER (master) și unul sau mai mulți target (slave)
! Masterul inițiază comunicarea

① SDA (Serial data) → transmite bit cu bit datele, fie de la master la slave fie invers
② SCL (serial clock) → masterul generează semnalul de ceas
Fiecare slave are o adresa de 7 biți (sau 10 biți)

Pull-up → liniile sunt trase în sus (High) prin rezistențe, dispozitivele „trag" în jos (LOW) pentru a transmite

① Half-duplex -> nu trimiți și primești simultan (folosești același fir SDA)
② Cine semnează -> master și slave folosesc același ceas
③ Pe RP2350 sunt două periferice I2C (I2C0, I2C1)

```
let tx_buf = [0x90u8]
i2c.write(0x5E, &tx_buf).unwrap(); // .unwrap() -> pentru a opri la eroare

// altfel e ceilalți: let mut rx_buf: [0u8; 7]
```

① new_blocking(_) -> creează un driver I2C care blochează firul de execuție până se termină transferul
② write(addr, &tx_buff) -> trimite tx_buf către dispozitivul cu adresa addr
③ read(addr, &mut rx_buf) -> citește în rx_buf

Curs 1 + 2 → GPIO

Operații bitwise fundamentale

// setarea bitului pe o poziție specificată

```
fn set_bit (register: usize, bit: u8) -> usize {
    register | (1 << bit)
}
```

// setarea mai multor biți odata

```
fn set_bits (register: usize, bits: usize) -> usize {
    register | bits
}
```

STERGEREA BIȚILOR

// Stergerea bitului pe poziția specificată

```
fn clear_bit (register: usize, bit: u8) -> usize {
    register & !(1 << bit)
}
```

// stergerea mai multor biți

```
fn clear_bits (register: usize, bits: usize) -> usize {
    register & !bits
}
```

// inversarea bitilor pe pozitia specificată

```
fn flip_bit (register: usize, bit: u8) -> usize {
    register ^ (1 << bit)
}
```

// extragerea valorilor din registru

```
const MASK: u32 = 0b0000_0000_0000_0000_0000_1111_1111_1111;
let extracted_bits = (large_id >> 20) & MASK;
```

Memory Mapped I/O (MMIO)

Conceptul de bază : → perifericele sunt accesate ca zone de memorie
→ fiecare registru are o adresă specifică
→ se folosesc pointeri pentru accesarea registrelor

GPIO pe RP2040

↳ Arhitectura GPIO
GPIO-urile pe RP2040 folosesc 3 PERIFERICE

① SIO (Single Cycle IO) - Controlează input / output

② IO BANK → Multiplexează funcțiile pinilor

③ PAD - Configurează parametrii electrici

Concepte importante despre semnale

- NOISE MARGIN → Spațiul de siguranță pentru semnale
- Pull-up / Pull-down → Rezistori pentru stări definite
- Level shifting → Pentru compatibilitatea între tensiuni diferite

Exercitii -> Operatii Bitwise

registru: 0b1000 | -> set_bit
bit = 2

Pas 1: calculez 1<<bit

1<<2 ?

Scriu 1 in binar = 0001
Shift left cu 2 pozitii: 0100

Deci 1<<2 = 0b0100

Pas 2: Calculez registru | (1<<bit)

registru = 0b1000
1<<bit = 0b0100

Operatia OR = 0b1100

Rezultat = 0b1100

CLEAR BIT

registru = 0b1100, bit = 2

Pas 1: Calculez 1<<bit

1 << 2 = 0b0100 (din ex anterior)

Pas 2: Calculez ! (1<<bit)

0b0100 NOT = 0b1011

Pas 3: Calculez registru & ! (1<<bit)

registru = 0b1100
!(1<<bit) = 0b1011

AND = 0b1000

Rezultat: 0b1000

Extragerea de biți

```
┌──────────────────────────────────────────────────┐
│ EXERCIȚIU: Extragere porțiune din ID             │
│ ID = 0b1100_1010_1111_1100_0000_1111_0110_1101   │
│ Vreau să extrag biții 31:20                      │
└──────────────────────────────────────────────────┘

Pas 1: Desenez ID-ul cu poziții
Pozițiile: 31 30 29 28 27 26 25 24 23 22 21 20 19 18...
ID:        1  1  0  0  1  0  1  0  1  1  1  1  1  1...

Zona pe care o vreau (31:20):
 1  1  0  0  1  0  1  0  1  1  1  1

Pas 2: Shift right cu 20 poziții
ID >> 20 = ?

Original (32 biți):
1100_1010_1111_1100_0000_1111_0110_1101

După >> 20 (umplu cu 0 la stânga):
0000_0000_0000_0000_0000_1100_1010_1111

Pas 3: Aplic masca
MASK = 0b0000_0000_0000_0000_0000_1111_1111_1111

(ID >> 20):  0000_0000_0000_0000_0000_1100_1010_1111
MASK:        0000_0000_0000_0000_0000_1111_1111_1111
             ____________________________________
Rezultat:    0000_0000_0000_0000_0000_1100_1010_1111

REZULTAT FINAL: 0b1100_1010_1111 (biții 31:20)
```

Interrupts, PWM, ADC → Curs 3

① Interrupts (Întreruperi)

- **Polling vs Interrupts**

① Polling = verificare periodică (consumă energie)

② Interrupts = răspuns la evenimente (eficient energetic)

- **Interrupt Controller**: primește semnale de la periferice și setează flag-uri pentru procesor
- **ISR (Interrupt Service Routine)**: funcția executată când e generată o întrerupere

② Timers și counters

→ alarme → pt măsurarea timpului exact
→ watchdog → interval maxim de verificări
→ control automat → PID, interval fix pentru rezultate optime
→ controlul motoarelor și PWM

COMPONENTE PRINCIPALE

- CLOCK SOURCE → cristal extern (XOSC) sau oscilator intern (ROSC)
- FREQUENCY DIVIDER → împarte frecvența de ceas
- COUNTER → incrementează un registru la fiecare ciclu de ceas

① SysTick (ARM CORTEX-M)

→ decrementează SYST_CUR la fiecare μs
→ când SYST_CUR = 0 → declanșează excepția SysTick
→ FRECVENȚĂ: $f = \frac{1}{SYST_RVR} \cdot 1\,000\,000\ Hz$

② RP2350 Timers

→ 2 timere (Timer0, Timer1) → fiecare cu 4 alarme
→ 64 bit counter → incrementează la fiecare μs
→ practic monoton (nu poate da overflow)

③ PWM (Pulse Width Modulation)

Principiul PWM
↳ generează semnale analogice folosind ieșiri digitale
↳ DUTY CYCLE → procentul timpului când semnalul este High
↳ Frecvența → numărul de repetări pe secundă

Formule PWM

① $f = \frac{1}{period}$ [Hz]

② $duty_cycle = \frac{time_on}{period} \cdot 100\%$

③ pentru RP2350

$$f = \frac{f_{clock}}{(divider \cdot (top+1)) \cdot (PHL_CORRECT+1))}$$

Utilizam PWM pentru:
- → controlul luminozitatii LED-urilor
- → controlul vitezei motoarelor
- → controlul unghiului servo-motoarelor

④ Semnale

Digital vs Analog

→ Analog → semnale reale, continue

→ Digital → reprezentare numerică, nivele discrete (0/1)

→ Noise Margin → Toleranta la zgomot pentru mentinerea starii logice

- Avantajele sistemelor digitale
  - ① Semnalul se reconstruieste după fiecare poartă logică
  - ② Margine de eroare acceptabila
  - ③ Informatia se păstrează după multiple "treceri"

⑤ ADC (Analog to Digital Converter)

Parametrii principali
- ① Sampling Rate: Frecventa de esantionare (Hz)
- ② Resolution: Nr de biti pt reprezentare
- ③ Nyquist-Shannon Theorem: $f_sampling > 2 \cdot f_max_signal$

- RP2350 ADC
  - → 4-8 canale, 500 KHz sampling rate, 12 biti resolution
  - → V_max = 3.3 V
  - → SAR (Successive Approximation Register) architecture

! Formula pentru temperatura interna

$$t = 27 - \frac{(V_input_4 - 0.706)}{0.001721} \ [°C]$$

Exercitii

① Calcule SysTick

Calculeaza valoarea SYST_RVR pentru o intrerupere la 5 secunde

Interval dorit = 5 000 000 µs
SYST_RVR = 5 000 000

② Calcularea unui timer pentru ceos real-time

Problemă: Pentru ATmega 328P (12 MHz), calculeaza OCR1A pentru 1 secundă cu prescaler 256

$$OCR1A = \frac{F_CPU}{prescaler} - 1$$

$$OCR1A = \frac{12\,000\,000}{256} - 1 = 46874$$

③ Calcule PWM

Problema: Pentru RP2350, calculează frecventa PWM cu:

top = 1000
divider = 125
fclock = 125 MHz
phase_correct = 0

$$f = \frac{fclock}{divider \cdot (top+1)}$$

$$f = \frac{125\,000\,000}{125 \cdot 1001} = 1000\,Hz$$

④ Calcule ADC

Problemă: Converteste valoarea ADC in tensiune

Valoare ADC = 2048
Resolution = 12 biti (4096 nivele)
V_ref = 3.3 V

$$Voltage = \frac{ADC_value \cdot V_ref}{2 \cdot resolution}$$

$$Voltage = \frac{2048 \cdot 3.3}{4096} = 1.65\,V$$

⑤ Calcule Duty Cycle

Problemă: Pt o frecventa PWM de 1 KHz si duty cycle de 25%, calculeazĂ timpul ON si OFF

$$Period = \frac{1}{1000} = 1\,ms$$

Time_ON = 25% · 1 ms = 0,25 ms
Time_OFF = 75% · 1 ms = 0,75 ms

⑥ Calcule NOISE MARGIN

Problemă: Calculează noise margin pentru:

$V_{OH} = 4.5V$

$V_{OL} = 0.5V$

$V_{IH} = 3.5V$

$V_{IL} = 1.5V$

$NM_{high} = V_{OH} - V_{IH} = 4.5 - 3.5 = 1.0V$

$NM_{low} = V_{IL} - V_{OL} = 1.5 - 0.5 = 1.0V$

⑦ Calcule Sampling Rate (Nyquist)

Problemă: Pentru un semnal cu $f_{max} = 20 KHz$, care este frecvența minimă de eșantionare?

$f_{sampling_min} = 2 \cdot f_{max} = 2 \cdot 20,000 = 40000 Hz$

$= 40 KHz$

Curs Async → Curs 4

① Concurența Preemptive vs Cooperative

Concurența Preemptive

- CARACTERISTICI
  - MCU-urile sunt de obicei single core
  - task-urile în paralel necesită un OS
  - task-urile pot fi suspendate oricând
  - schimbarea task-urilor este costisitoare
  - problemă: task-urile cu mult I/O fac timpul de switching mai lung decât procesarea efectivă

- Proces
  - OS primește alarm de la Systick
  - salvează starea task-ului curent
  - restaurează starea următorului task
  - task-ul așteaptă hardware-ul

Concurența Cooperative

- Caracteristici
  - task-urile nu pot fi întrerupte (excepțând ISR)
  - hardware-ul funcționează asincron
  - task-urile cooperează - cedează MCU-ul în timp ce așteaptă hardware-ul
  - nu este nevoie de OS - totul într-un gg flow
  - nu (J) penalizare pentru salvarea / restaurarea stării

- Proces
  - scheduler verifică dacă (J) task ready
  - task-ul face request la hardware
  - task-ul folosește .await pentru a ceda controlul
  - scheduler rulează alt task
  - hardware trimite un event când se termină

② Asynchronous Executor (Embassy)

Task-uri:

① #[embassy_executor::main] → porneste scheduler-ul Embassy, și definește task-ul principal

② #[embassy_executor::task] → definește un task nou

③ pool_size - optional, definește câte task-uri identice pot fi spawned

❗
① Task-urile trb să folosească .await în loop-uri
② Fără .await, async function-urile nu se execută
③ Dacă un task nu folosește .await, blochează scheduler-ul

Asynchronous Executor: → așteaptă evenimente când toate task-urile așteaptă
→ după execuția unui ISR, întreabă fiecare task dacă poate executa
→ dacă un task se execută, continuă până la .await
→ dacă un task nu folosește .await, executorul nu mai rulează alte task-uri

● **Priority Tasks** → task-urile de priorități rulează în ISR-uri
→ task-urile de priorități mică sunt întrerupte
→ se folosesc InterruptExecutor pentru priorități înaltă/medie
→ StaticCell<Executor> pentru priorități joasă

● Future type → promise în alte limbaje

Implementarea unui Future

```rust
enum SleepStatus {
    SetAlarm,
    WaitForAlarm,
}

impl Future for Sleep {
    type Output = ();
    fn poll(&mut self) -> Poll<Self::Output> {
        match self.status {
            SleepStatus::SetAlarm => {
                ALARM.set_alarm(self.timeout);
                self.status = SleepStatus::WaitForAlarm;
                Poll::Pending
            }
            SleepStatus::WaitForAlarm => {
                if ALARM.expired() {
                    Poll::Ready(())
                } else {
                    Poll::Pending
                }
            }
        }
    }
}
```

→ Rust recunoaște funcțiunile async în Future-uri
→ Nu știe cum să le execute → executor-urile sunt biblioteci în third party

→ Apelarea unei funcții async returnează Future-ul, dar nu îl execută

Synchronization Primitives

Tipuri de Mutex

- **NoopMutex** - pentru date partajate între task-uri din același executor
- **CriticalSectionMutex** - pentru date partajate între executori multipli, ISR-uri și core-uri
- **ThreadModeMutex** - pentru date partajate între task-uri din executori de prioritate joasă

Blocking Mutex

- Nu permite `.await` în timp ce mutex-ul este ținut
- Folosește `RefCell` pentru interior mutability

Async Mutex

- Permite `.await` în timp ce Mutex-ul este ținut
- Va elibera Mutex-ul în timpul `.await`-ului

Channels

Tip	Descriere
Channel	MPMC - fiecare mesaj este primit de un singur consumer
PriorityChannel	MPMC cu prioritate - mesajele cu prioritate mai mare sunt mutate în față
Signal	Semnalizează ultima valoare către un singur consumer
PubSubChannel	Broadcast - fiecare mesaj este primit de toți consumerii

Exercitii

1

Un sistem embedded are următoarele task-uri:

- **TaskA**: execută 3ms, apoi await 20ms (ciclu total: 23ms)
- **TaskB**: execută 5ms, apoi await 30ms (ciclu total: 35ms)
- **TaskC**: execută 2ms, apoi await 10ms (ciclu total: 12ms)

a) Calculați utilizarea CPU pentru fiecare task în procente. **b)** Determinați timpul maxim de idle în primele 140ms. **c)** La ce moment se vor suprapune execuțiile TaskA și TaskB pentru prima oară?

Task A : 3ms exec + 20 ms await = 23ms ciclu

Task B = 35ms

Task C = 12ms

a) Task A = $\frac{3\,ms}{23\,ms} \cdot 100\% = 13.04\%$

Task B = $\frac{5\,ms}{35\,ms}$ $100\% = 14.29\%$

Task C = $\frac{2\,ms}{12\,ms} \cdot 100\% = 16.64\%$.

$\Rightarrow$ Total CPU = $13.04\% + 14.29\% + 16.67\%$

$= 44\%$

(b)

Task A executa: $\frac{140}{23} = 6.09$ -> 6 executii complete => $6 \cdot 3 = 18$ ms

Task B: $\frac{140}{35} = 4$ -> 4 executii complete => $4 \cdot 5 = 20$ ms

Task C: $\frac{140}{12} = 11\,67$ -> 11 executii complete => $11 \cdot 2$ ms $= 22$ ms

Timpul total de executie: $18 + 20 + 22 = 60$ ms

Total idle time: $140 - 60 = 80$ ms

Timpul maxim continuu idle: 10-12 ms (intre cicluri)

(c) Suprapunere task A si task B

TASK A: 0, 23, 46, 69, 92, 115

TASK B: 0, 35, 70, 105

Suprapunere la $t = 70$ ms

Task A începe la $t = 69$ ms $(69 + 3 = 72)$

Task B incepe la $t = 70$ ms $(70 + 5 = 75)$

Suprapunere: 70-72 ms (2 ms)

(2)

Un sistem cu priorități are:

- **HIGH**: execută la fiecare 25ms, durată 4ms
- **MEDIUM**: execută la fiecare 60ms, durată 8ms
- **LOW**: task continuu care execută 1ms apoi await 1ms

a) Calculați utilizarea CPU pentru fiecare prioritate în 300ms. **b)** De câte ori va fi întrerupt LOW priority în 300ms?

→ High (25ms → perioadă, 4ms - execuție)

→ execuții : $\frac{300}{25}$ (CPU în 300 ms) = 12 execuții

→ timp total : 12 · 4 = 48 ms

→ utilizare : $\frac{48}{300} = 16\%$

→ Medium (60 ms → perioadă, 8 ms - execuție)

→ execuții : $\frac{300}{60}$ (CPU în 300 ms) = 5 execuții

→ timp total : 5 · 8 = 40 ms

→ utilizare : $\frac{40}{300} = 13.33\%$

→ LOW (1 ms → perioadă + 1ms await = 2ms ciclu

→ timp disp : 300 (CPU în 300 ms) − 48 − 40 = 212 ms

→ execuții LOW : $\frac{212}{2}$ = 106 execuții · 1 = 105 ms

→ utilizare : $\frac{106}{300} = 35.33\%$

Total : $16\% + 13.33\% + 35.33\% = 64.66\%$

(b) Întreruperi LOW în 300 ms 4 : 2 = 2 (2 cicluri la LOW)

High : 12 execuții · 2 întreruperi / execuție = 24 întreruperi

, 8 : 2 = 4 (2 cicluri la LOW)

Medium : 5 execuții · 4 întreruperi / execuție = 20 întreruperi

(+)

Total = 44 întreruperi

3

Un channel primește mesaje de la 3 produceri:

- **Producer1**: 1 mesaj la fiecare 50ms
- **Producer2**: 1 mesaj la fiecare 80ms
- **Producer3**: 1 mesaj la fiecare 120ms

Consumer-ul procesează 1 mesaj la fiecare 40ms.

a) Calculați rata totală de producție și consum pe secundă. **b)** Determinați dimensiunea minimă de buffer pentru 5 secunde. **c)** La ce moment se va umple buffer-ul dacă are dimensiunea 10?

Producer 1 : $\frac{1000\ ms}{50\ ms} = 20\ msg/s$

Producer 2 : $\frac{1000\ ms}{80\ ms} = 12.5\ msg/s$

Producer 3 : $\frac{1000\ ms}{120\ ms} = 8.3\ msg/s$

(+)

Total $= 40.83\ msg/s$

Consumer $= \frac{1000\ ms}{40\ ms} = 25\ msg/s$

Net accumulation : $40.83 - 25 = 15.83\ msg/s$

(b) Buffer minim pt 5S

In 5 secunde se vor acumula : $15.83 \cdot 5 = 79.15$ mesaje
Buffer minim necesar : 80 mesaje

(c) Buffer 10 se umple după

Rata acumulare : 15.83 mg/s
Timp pt 10 mesaje : $\frac{10}{15.83} = 0,63\ s = 630\ ms$

4. Un PriorityChannel are:

- **Mesaje HIGH**: 1 la fiecare 100ms (prioritate 1)
- **Mesaje NORMAL**: 3 la fiecare 200ms (prioritate 2)

Procesarea: HIGH în 10ms, NORMAL în 15ms.

Calculați timpul maxim de așteptare pentru un mesaj NORMAL în steady state.

Mesaj high : 1 la 100 ms (procesare 10 ms)

Mesaj normal : 3 la 200 ms (procesare 15 ms)

Worst case normal

↳ un mesaj normal ajunge exact când începe procesarea HIGH
↳ HIGH are întotdeauna prioritate

În 200 ms : 2 mesaje high (20 ms) + 3 normale (45ms) = 65 ms

Așteptare maxim normal: 10 ms (High) + 45 ms (normal) = 55 ms

5. Care din următoarele cauzează deadlock în Embassy?

- a) Două AsyncMutex luate în ordine diferită
- b) BlockingMutex ținut în timpul unui .await
- c) Channel plin cu sender care așteaptă
- d) PriorityChannel cu prioritate negativă

6. Pentru un sistem cu 5 task-uri identice (2ms exec + 18ms await), utilizarea CPU este:

- a) 50%
- b) 10%
- c) 40%
- d) 25%

$$\frac{(5 \cdot 2ms)}{2+18} = \frac{10ms}{20ms} = 0,5 = 50\%$$

7. Un Future este poll()-at de 15 ori înainte să returneze Ready. Câte stări intermediare minimale are?
Justificați cu un exemplu.

Minum 14 stări intermediare

exp : ConnectionFuture cu multiple retry steps

(8) Explicați diferența între `Poll::Pending` și `Poll::Ready(())` în contextul unui TimerFuture. Când se folosește fiecare?

Poll::Pending → timer încă nu a expirat, continuă așteptarea

Poll::Ready → timer expirat, task poate continua

Curs 5 → UART & SPI

① DMA (Direct Memory Acces)

- descarcă MCU-ul de operații memory-to-memory
- transferă date direct între memorie și periferice fără intervenția CPU
- eliberează CPU pt alte operații
- ridică interrupt când transferul e complet

Componente: → canale multiple
↳ fiecare canal are: Source Address, Destination Address, Length

② UART (Universal Asynchronous Receiver and Transmitter)

Caracteristici esențiale

1. Conectare: 2 fire independente (TX & RX), cross-connected
2. Asincron: nu (!) ceas partajat între dispozitive
3. Duplex: poate primii & trimite simultan
4. Conectează doar două dispozitive
5. Viteză: Obișnuit 9600 sau 115200 hard

STRUCTURA UNUI FRAME

START | PAYLOAD (5-9 biți) | PARITY (opțional) | STOP (1-2 biți)

③ FORMULA BAUD RATE

$$baud_rate = \frac{f_clock}{divider \cdot (1 + payload_bits + parity_bits + stop_bits))}$$

④ Tipuri UART

- TTL: 0-3.3V sau 0-5V (distante scurte)
- RS232: -12V la 12V (distante mari)
- RS485: Industrial, semnal diferențială

⑤ Componente interne

- RX FIFO -> buffer pt receptie
- TX FIFO -> buffer pt transmisie
- Interrupts: RX IRQ, TX IRQ
- Enable bits: Enable RX, Enable TX, Enable IRQ

⑥ SPI (Serial Peripheral Interface)

- Conectare: 1 main (master) + mai multi slaves
- Sincron: clockul e generat de main
- Duplex: Transmisie semiultan în ambele directii
- Viteză: fără limită teoretică (limitată de hardware)

Fire → ① MOSI -> Master out slave in

② MISO -> Master in Slave out

③ CLK: Clock generat de main

④ CS: Chip Select (1 fir pe slave) - nu face parte din SPI

⑤ Total: 3 + n fire (n = nr slave)

Moduri SPI (CPOL + CPHA):

Mode	CPOL	CPHA	Citire pe	Scriere pe
0	0	0	Rising edge	CS activate/clock edge
1	0	1	Rising edge	Clock edge
2	1	0	Falling edge	CS activate/clock edge
3	1	1	Falling edge	Clock edge

Proces de transmisie:

1. Main activează CS (LOW)
2. Main și sub pun primul bit pe linii
3. Main pornește clock-ul
4. Pe fiecare clock edge: citire și scriere de biți
5. Main oprește clock-ul
6. Main dezactivează CS (HIGH)

⑦ DAISY CHAINING

↳ toți slave sunt activi semultan

↳ datele trec prin rând: MAIN → Sub 1 → Sub 2 → ... → Sub n → Main

⑦ Senzori Analogici vs Digitali

- Senzori analogici → doar transductorul (elementul de măsurare)
  → OUTPUT: tensiune analogică
  → Necesită: ADC + filtrare zgomot în MCU

- Senzori digitali → transductor + ADC + MCU intern + interfață digitală
  → OUTPUT: date digitale prin bus (SPI/I2C)
  → Exp: BMP280 (presiune / temperatură)
    ↳ Registre importante: temp_xlsb (0xFC), press_lsb (0xF8), ctrl_meas (0xF4)
    ↳ Citire: bit 7 = 1 în adresă
    ↳ Scriere: bit 7 = 0 în adresă

UART:

```rust
// Sincron
let mut uart = uart::Uart::new_blocking(p.UART0, p.PIN_0, p.PIN_1, config);
uart.blocking_write("Hello".as_bytes());
uart.blocking_read(&mut buf);

// Asincron
let mut uart = uart::Uart::new(p.UART0, p.PIN_0, p.PIN_1, Irqs, p.DMA_CH0, p.DMA_CH1, con
uart.write("Hello".as_bytes()).await;
uart.read(&mut buf).await;
```

SPI:

```rust
// Sincron
let mut spi = Spi::new_blocking(p.SPI1, clk, mosi, miso, config);
spi.blocking_transfer_in_place(&mut buf);

// Asincron
let mut spi = Spi::new(p.SPI1, clk, mosi, miso, p.DMA_CH0, p.DMA_CH1, config);
spi.transfer(&mut rx_buf, &tx_buf).await;
```

EXERCIȚII

Exercițiul 1: Calcul Baud Rate UART

Enunț: Un MCU cu frecvența de ceas de 48 MHz folosește UART cu configurația 8P1 (8 biți payload, parity, 1 stop bit). Care trebuie să fie valoarea divider-ului pentru a obține un baud rate de 115200?

$f_{clock} = 48\,000\,000$ Hz

$baude_rate = 115200$

$payload_bits = 8$

$parity_bits = 1$

$stop_bits = 1$

$$baud_rate = \frac{f_{clock}}{divider \cdot (1 + payload_bits + parity_bits + stop_bits))}$$

$$115200 = \frac{48000000}{divider \cdot (1+8+1+1)}$$

$$115200 = \frac{48\,000\,000}{divider \cdot 11} \Rightarrow divider = \frac{48\,000\,000}{115200 \cdot 11}$$

$$divider = 37{,}8 \Rightarrow divider \simeq 38$$

② **Exercițiul 2: Analiza Diagramă UART**

Enunț: Analizează această secvență UART și determină: a) Configurația (XYZ format) b) Valoarea byte-ului transmis (în hexazecimal) c) Baud rate-ul dacă un bit durează 104 µs

```
Idle |START| b0| b1| b2| b3| b4| b5| b6| b7|PARITY|STOP|
 HIGH| LOW | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |  1   |HIGH|
```

ⓐ Configurația

8 biți payload (b0 - b7)

1 bit parity

1 bit stop

Format : 8P1

ⓑ Valoarea byte-ului

Biții se transmit LSB first : $b0=1$, $b1=0$, $b2=1$, $b3=1$, $b4=0$, $b5=0$, $b6=1$, $b7=0$

Valoare binară : 01001101 (MSB la stânga)

Valoare hex : 0x4D

ⓒ Baud rate

1 bit = 104 µs

$$\text{Baud rate} = \frac{1}{104 \cdot 10^{-6}} = 9,615 \text{ baud} = 9600 \text{ baud}$$

3

Enunț: Ai un sistem cu 1 MCU (main) și 3 senzori (subs) conectați prin SPI. Desenează schema de conectare și explică: a) Câte fire sunt necesare în total? b) Care este procedura pentru a citi de la sub-ul 2? c) Ce se întâmplă cu sub-urile 1 și 3 când main comunică cu sub-ul 2?

(a) Fire necesare

- MOSI -> 1 fir (partajat)
- MISO -> 1 fir (partajat)
- CLK -> 1 fir (partajat)
- CS -> 3 fire (câte unul pe sleve)

(b) Procedură pt sub 2

① Master activează CS2 (LOW)

② Master pune primul bit pe MOSI, slove pe MISO

③ Master pornește clock-ul

④ Transmite biți pe clock edges

⑤ Master oprește clock-ul

⑥ Master dezactivează CS2 (High)

(c) Slove 1 și 3

-> CS1 și CS3 rămân inactive

-> slove 1 și slove 3 se deconectează de pe liniile MOSI și MISO

-> slove 1 și slove 2 ignoră clock-ul

4

Enunț: Pentru senzorul BMP280: a) Scrie secvența SPI pentru a citi registrul temp_msb (adresa 0xFA) b) Scrie secvența SPI pentru a configura ctrl_meas (0xF4) cu valoarea 0b10001011 c) Explică diferența dintre bit-ul 7 în cele două operații

Rezolvare:

```
a) Citire temp_msb (0xFA):
   MOSI: [1 1 1 1 1 0 1 0] [0 0 0 0 0 0 0 0]
         |-----0xFA-----|   |--dummy byte--|
         bit7=1 (READ)

   MISO: [x x x x x x x x] [d7 d6 d5 d4 d3 d2 d1 d0]
         |--don't care--|   |----data value-----|

b) Scriere ctrl_meas (0xF4):
   MOSI: [0 1 1 1 0 1 0 0] [1 0 0 0 1 0 1 1]
         |-----0x74-----|   |----0x8B-----|
         bit7=0 (WRITE)

   MISO: ignorat

c) Diferența bit-ului 7:
   - Bit 7 = 1: operație de CITIRE
   - Bit 7 = 0: operație de SCRIERE
   Pentru scriere, adresa 0xF4 devine 0x74 (bit 7 = 0)
```

6

Exercițiul 6: Comparație UART vs SPI

Enunț: Completează tabelul de comparație:

Caracteristică	UART	SPI
Nr. dispozitive	?	?
Tip clock	?	?
Nr. fire	?	?
Viteza max	?	?
Complexitate	?	?

Rezolvare:

```
| Caracteristică | UART | SPI |
|----------------|------|-----|
| Nr. dispozitive | 2 | 1 main + N subs |
| Tip clock | Asincron | Sincron |
| Nr. fire | 2 | 3 + N |
| Viteza max | ~115200 baud | Fără limită teoretică |
| Complexitate | Simplă | Medie |
```

7

Enunț: Un sistem folosește DMA pentru a transfera 1024 bytes de la UART la memorie. Explică: a) Ce avantaje aduce DMA față de polling? b) Ce probleme pot apărea cu cache-ul? c) Când se va declanșa interrupt-ul?

Rezolvare:

a) Avantajele DMA:
- CPU-ul e liber să facă alte operații
- Transfer automat fără intervenția software
- Eficiență mare pentru transferuri mari de date
- Reduce latența de răspuns

b) Probleme cu cache-ul:
- DMA nu știe de cache-ul CPU
- Datele din cache pot fi diferite de cele din memorie
- Soluții: flush cache înainte de DMA,
  invalidate cache după DMA,
  setare regiune nocache

c) Interrupt-ul:
- Se declanșează când toți 1024 bytes au fost transferați
- CPU poate procesa datele sau porni următorul transfer

I2C & USB -> Curs 6

1) I2C (Inter-Integrated Circuit)

Caracteristici
- 2 fire: SDA (Serial Data) și SCL (Serial Clock)
- Master-Slave: Controller (master) inițiază comunicația, target (slave) răspunde
- Adrese: 7 sau 10 biți pt fiecare target
- Pull-up: Firele sunt mereu high prin PULL-UP, dispozitivele trag LOW
- Half-Duplex: Datele se transmit într-o singură direcție la un moment dat

- TRANSMISIE I2C: ① START CONDITION (SDA LOW, apoi SCL LOW)
  ② Adresa target + R/W bit
  ③ ACK, NACK de la target
  ④ Transfer date (cu ACK după fiecare byte)
  ⑤ STOP condition (SCL HIGH, apoi SDA HIGH)

- Moduri de viteză : ① STANDARD : 100 Kbit/s
  ② FAST : 400 Kbit/s
  ③ FAST+ : 1 Mbit/s
  ④ HIGH-SPEED: până la 3.4 Mbit/s

② USB 2.0 (UNIVERSAL SERIAL BUS)

Caracteristici
- > HOST-DEVICE : host-ul controlează comunicația
- > 4 fire: D+/D- (diferențial), Vcc, Gnd
- > 48 MHz clock
- > endpoints : până la 16 (IN/OUT)
  ↳ de la device la host
- > half-duplex

- Tipuri de pachete USB : 1 TOKEN : OUT, IN, SETUP
  2 DATA : DATA0, DATA1 (0-1024 bytes)
  3 HANDSHAKE : ACK, NACK, STALL

- MODURI DE TRANSFER : 1 CONTROL : Pentru configurare (SETUP -> DATA -> STATUS)
  2 BULK : Transfer mare, reliable dar fără garanție de timp
  3 INTERRUPT : Transfer periodic cu latență garantată
  4 ISOCHRONOUS : Bandwidth garantat, permite pierderea datelor

- ORGANIZAREA DISPOZITIVULUI : DEVICE - CONFIGURATION -> INTERFACE -> ENDPOINT

- EXERCIȚII

1

Exercițiul 1: Calculul vitezei I2C

Enunț: Un senzor BMP280 trimite 6 bytes de date prin I2C în mod Standard (100 kbit/s). Calculați timpul necesar pentru o transmisie completă incluzând: START, adresa (7 biți), R/W bit, ACK, 6 bytes de date cu ACK după fiecare byte, și STOP.

START : 1 bit

Adresa : 7 biți

$\frac{R}{W}$ = 1 bit

ACK după adresa : 1 bit

6 bytes date : 6 · 8 = 48 biți

ACK după fiecare byte : 6 biți

STOP : 1 bit

TOTAL BIȚI : 1 + 7 + 1 + 1 + 48 + 6 + 1 = 65 biți

TIMP : $\frac{65 \text{ biți}}{100\,000 \text{ biți/s}}$ = 0,65 ms

2

Exercițiul 2: Adrese I2C

Enunț: Într-un sistem I2C cu 5 senzori cu adresele 0x23, 0x44, 0x68, 0x76, 0x48, determinați: a) Ce valoare se trimite pe bus pentru a citi de la senzorul cu adresa 0x68? b) Ce valoare se trimite pentru a scrie la senzorul cu adresa 0x23?

a) Pentru citire $\frac{R}{W}$ = 1

0x68 << 1 <=> 1 = 0x80 <=> 0 = 0x81

b) Pentru scriere $\frac{R}{W}$ = 0

0x23 << 1 <=> 0 = 0x46

3)

Exercițiul 3: Registrul BMP280

Enunț: Pentru senzorul BMP280, scrieți secvența de cod Embassy pentru a configura registrul ctrl_meas (0xF4) cu valorile:

- osrs_t = 010 (oversampling temperatura × 2)
- osrs_p = 101 (oversampling presiune × 16)
- mode = 11 (mod normal)

Rezolvare:

```rust
const DEVICE_ADDR: u8 = 0x77;
const CTRL_MEAS_REG: u8 = 0xF4;

// osrs_t[7:5] = 010, osrs_p[4:2] = 101, mode[1:0] = 11
let value = 0b010_101_11; // = 0x5F

let buf = [CTRL_MEAS_REG, value];
i2c.write(DEVICE_ADDR, &buf).unwrap();
```

4)

Exercițiul 4: Pachete USB

Enunț: Un dispozitiv USB trimite un pachet DATA0 cu 32 bytes către host. Calculați: a) Dimensiunea totală a pachetului în biți b) Timpul de transmisie la Full Speed (12 Mbit/s)

a) Pachet DATA0

SYNC : 8 biți

PID : 8 biți (4 + 4 inversat)

DATE : $32 \cdot 8 = 256$ biți

CRC : 16 biți

EOP : 2 biți Total $= 8 + 8 + 256 + 16 + 2 = 290$ biți

b) $$\text{Timp} = \frac{290 \text{ biți}}{12\,000\,000 \text{ biți/s}} = 27.17 \ \mu s$$

6

Exercițiul 6: Calculul puterii USB

Enunț: Un dispozitiv USB declară în Configuration Descriptor că consumă bMaxPower = 75. Calculați puterea reală consumată în miliamperi și wați (la 5V).

bMAXPOWER = 2 mA unitati

Curent real = 75 · 2mA = 150 mA

PUTERE : 5V · 0.15A = 0.75 W

7

Exercițiul 7: Endpoint Address

Enunț: Un dispozitiv USB are următoarele endpoints:

- EP0 IN/OUT (control)
- EP1 IN (bulk)
- EP2 OUT (bulk)
- EP3 IN (interrupt)

Determinați valorile bEndpointAddress pentru fiecare endpoint.

Rezolvare:

- EP0 OUT: 0x00, EP0 IN: 0x80
- EP1 IN: 0x81
- EP2 OUT: 0x02
- EP3 IN: 0x83

8

Exercițiul 8: Comparație I2C vs USB

Enunț: Comparați avantajele și dezavantajele I2C față de USB pentru conectarea unui senzor de temperatură.

Rezolvare: I2C: ✓ Simplu (2 fire) ✓ Consum redus ✓ Ideal pentru senzori ✗ Viteză limitată ✗ Distanță scurtă

USB: ✓ Viteză mare ✓ Alimentare inclusă ✓ Standard universal ✗ Complex ✗ Consum mai mare ✗ Overhead pentru senzori simpli

Concluzie: Pentru un senzor simplu de temperatură, I2C este mai potrivit.

Curs 7 → Certificări de vânzare

① Certificări : → simboluri distincte recunoscute internațional

→ obligatorii pentru vânzarea / distribuirea produselor în anumite țări

→ pot fi verificate pre-certificare sau autocertificate cu responsabilitate asumată

→ tipuri : → individuală
↳ de design
↳ de companie
↳ de persoană

② Restricții pentru certificări

↳ în funcție de zona geografică : ① fiecare țară / regiune impune obligativitatea anumitor certificări

② Exp: CE (UE), FCC (SUA), UL (SUA / Canada)

③ **În funcție de tipul aplicației:**

- **Uz general**: încărcătoare, televizoare, frigidere
- **Iluminat**: becuri LED, economice, halogen
- **Uz industrial**: PLC-uri, drivere motoare, surse tensiune
- **Măsurători laborator**: multimetre, osciloscoape
- **Măsurători industriale**: sisteme achiziție date
- **Uz medical**: EKG, RMN

ϟ **În funcție de caracterul critic:**

- **Non-critice**: defectarea nu pune în pericol viețile/mediul
- **Critice**: defectarea poate cauza distrugeri materiale, pericol vieți, distrugerea mediului

În funcție de amplasament:

- Zone explozive (ATEX)
- Aplicații militare, medicale, automotive, industriale
- Zone cu grad ridicat de praf/umiditate (IP)

⑤ Standardul IP (International Protection)

Format: IP X Y

↳ Prima cifra (X): → rezistență la prof (0-6)
0 -> fără protecție
6 -> dust tight (protecție completă)

↳ A doua cifra (Y): → rezistența la apa (0-9K)
↳ 0 -> fără protecție
↳ 8 -> imersiune > 1m adâncime

⑥

4. Echipamente de Uz General

UE + Țările Nordice:

- **CE Mark**: obligatoriu pentru comercializarea în EEA
- Asigură protecția consumatorului
- Verifică: EMI (nu interferează), EMC (nu fi interferat)

SUA:

- **FCC**: pentru dispozitive cu oscilații >9kHz
- Clase: A (industrial - mai permisivă), B (home-use - mai restrictivă)
- **UL**: pentru dispozitive la priză

⑦

5. Testarea EMI/EMC

Camere anecoice:

- Spații speciale fără reflexii de câmp electromagnetic
- Similare camerelor de studio audio
- Testare cu antene speciale (near field) sau antene de cameră

6. Directive de Protecție a Mediului

ROHS (Restriction of Hazardous Substances):

- Restricționează substanțele periculoase
- Din 2017: obligatoriu și pentru instrumentele industriale de monitoring

WEEE (Waste Electrical and Electronic Equipment):

- Gestionarea deșeurilor electronice
- Simbolul coșului de gunoi barat

⑧

7. Dispozitive Medicale

Europa - 5 clase:

- Class I (Non-Sterile, Non-Measuring)
- Class I (Sterile, Measuring)
- Class IIa, IIb, III (risc crescând)

SUA - 4 clase:

- Class I: risc foarte mic
- Class II: risc mic
- Class III: risc relativ ridicat
- Class IV: risc letal

9

8. ATEX (Zone Explozive)

Zone pentru gaze/vapori:

- Zone 0, 1, 2 (prezența continuă/intermitentă/anormală)

Zone pentru praf combustibil:

- Zone 20, 21, 22

Certificarea ATEX:

- Individuală pentru fiecare dispozitiv
- NU există auto-certificare
- În România: INSEMEX

Exercitii

1

Exercițiul 1 - Identificarea Certificărilor

Enunț: Identificați certificările necesare pentru următoarele produse și piețele de destinație: a) Un încărcător de telefon pentru piața UE b) Un multimetru de laborator pentru SUA c) Un senzor de temperatură pentru o rafinărie (zona explozivă)

Rezolvare: a) **Încărcător telefon UE**: CE Mark (obligatoriu), posibil RoHS/WEEE pentru protecția mediului b) **Multimetru SUA**: FCC (pentru EMI/EMC), UL (pentru siguranța la priză) c) **Senzor rafinărie**: ATEX (pentru zona explozivă), CE Mark (pentru UE), certificare individuală obligatorie

2

Exercițiul 2 - Clasificarea IP

Enunț: Un dispozitiv electronic trebuie să funcționeze în condiții de praf moderat și să reziste la stropi de apă. Care este ratingul IP minim necesar?

Rezolvare:

- Pentru praf moderat: IP5X (dust protected - praful nu interferează cu funcționarea)
- Pentru stropi de apă: IPX3 (spraying water)
- **Rating IP minim necesar: IP53**

3

Exercițiul 3 - Zone ATEX

Enunț: Într-o fabrică de făină, unde praful combustibil este prezent intermitent în timpul proceselor normale de lucru, ce zonă ATEX se aplică și ce certificări sunt necesare?

Rezolvare:

- Praf combustibil prezent intermitent = **Zona 21**
- Certificări necesare:
  - **ATEX** (obligatoriu pentru zona explozivă)
  - **CE Mark** (pentru UE)
  - Certificare individuală pentru fiecare echipament
  - NU există posibilitatea de auto-certificare

4

Exercițiul 4 - Dispozitive Medicale

Enunț: Clasificați următoarele dispozitive medicale conform standardelor europene și americane: a) Termometru digital b) Aparat de dializă c) Stimulator cardiac

Rezolvare: a) **Termometru digital:**

- Europa: Class I (Non-Sterile, Non-Measuring)
- SUA: Class I (risc foarte mic)

b) **Aparat de dializă:**

- Europa: Class IIb (risc mediu-ridicat)
- SUA: Class II (risc mic controlat)

c) **Stimulator cardiac:**

- Europa: Class III (risc ridicat)
- SUA: Class IV (risc letal)

5

Exercițiul 5 - EMI/EMC

Enunț: Un dispozitiv electronic funcționează la 50 MHz și va fi comercializat în SUA pentru uz casnic. Ce certificări FCC sunt necesare și ce clasă se aplică?

Rezolvare:

- Frecvența 50 MHz > 9 kHz → **FCC obligatoriu**
- Uz casnic → **Clasa B** (mai restrictivă decât Clasa A industrială)
- Testare necesară în cameră anecoică
- Echivalent european: CISPR 22

6

Exercițiul 7 - Timeline Certificări

Enunț: Un produs trebuie certificat pentru piața UE (CE + ATEX). Duratele estimate sunt:

- CE Mark: 6 săptămâni
- ATEX: 12 săptămâni
- Procesele pot rula în paralel pentru primele 4 săptămâni

Calculați timpul minim total pentru certificare.

Rezolvare:

- Primele 4 săptămâni: procese în paralel
- CE Mark: 6 - 4 = 2 săptămâni rămase
- ATEX: 12 - 4 = 8 săptămâni rămase
- **Timp total minim: 4 + max(2, 8) = 4 + 8 = 12 săptămâni**

Curs 8 -> Wifi

1. OSI Network Stack (Stiva de Rețea OSI)

Cele 7 straturi:

1. **Physical (L1)** - hardware, transmite/primește date pe mediul fizic
2. **Data Link (L2)** - hardware și driver, comunicare directă între dispozitive conectate
3. **Network (L3)** - driver, rutare către dispozitive neconectate direct
4. **Transport (L4)** - conectează aplicațiile la stiva de rețea
5. **Session (L5)** - nu se folosește
6. **Presentation (L6)** - nu se folosește
7. **Application (L7)** - aplicația

Principiu: Fiecare strat comunică cu omologul său de pe alte dispozitive.

2. Wi-Fi

Caracteristici:

- Protocol L2 (Data Link)
- Frecvențe: 2.4 GHz și 5 GHz
- Componente: AP (Access Point) și Device

Tipuri de securitate:

- **Open** - toată lumea primește comunicația
- **WEP** - toate datele criptate cu aceeași cheie
- **WPA 1/2/3 Personal** - fiecare dispozitiv are cheie diferită cu AP-ul
- **WPA 1/2/3 Enterprise** - autentificare cu username/password sau cheie privată

Tipuri de dispozitive de rețea:

- **Integrated** - radio integrat în MCU, controlat software
- **Discrete** - MCU conectat la dispozitiv Wi-Fi extern via UART/SPI/I2C

3. TCP/IP Stack

Straturile TCP/IP:

- **Physical** - transmitere fizică
- **Data Link** - frame-uri MAC (adrese MAC)
- **Network** - pachete IP (adrese IPv4/IPv6)
- **Transport** - TCP/UDP cu porturi
- **Application** - aplicația

Data Link Layer:

- Folosește adrese MAC
- Format frame: `Destination MAC | Source MAC | Payload | CRC`

Network Layer:

- Internet Protocol (IP)
- IPv4: 32 biți, IPv6: 128 biți
- Format: `MAC Headers | Source IP | Destination IP | Payload | CRC`

Transport Layer:

- **TCP** - flux de date, garantează livrarea
- **UDP** - "fire and forget", best effort
- Folosește porturi pentru identificarea aplicațiilor

4. Raspberry Pi Pico W

Specificații:

- Wi-Fi chip: CYW43439 (Infineon)
- Conectare: SPI/PIO0
- Wi-Fi 4 (802.11n), 2.4 GHz
- WPA 3, SoftAP (4 clienți)
- BLE 5.2, Bluetooth Classic

Embassy Framework:

- **smoltcp** - stack TCP/IP embedded (no_std)
- Suportă: IPv4, IPv6, IGMP, TCP, UDP, DHCP, DNS

5. Protocoale de Aplicație

- **MQTT** - MQ Telemetry Transport (publish/subscribe)
- **CoAP** - Constrained Application Protocol (HTTP simplificat binar)
- **DNS** - Domain Name System
- **DHCP** - Dynamic Host Configuration Protocol

Exercitii

1

Exercițiul 1: OSI Model (5 puncte)

Explicați rolul fiecărui strat din modelul OSI și dați un exemplu de protocol pentru fiecare strat folosit.

Rezolvare:

- **L1 Physical**: Transmiterea electrică/optică a biților (ex: Ethernet 100Base-T, Wi-Fi radio)
- **L2 Data Link**: Frame-uri cu adrese MAC, control acces mediu (ex: Ethernet, Wi-Fi 802.11)
- **L3 Network**: Rutare cu adrese IP (ex: IPv4, IPv6, ICMP)
- **L4 Transport**: Controlul fluxului și porturile (ex: TCP, UDP)
- **L5 Session**: Gestionarea sesiunilor (ex: NetBIOS, RPC)
- **L6 Presentation**: Criptare, compresie (ex: SSL/TLS, JPEG)
- **L7 Application**: Interfața utilizator (ex: HTTP, FTP, SMTP)

2

Exercițiul 2: Wi-Fi Security (4 puncte)

Comparați securitatea WEP vs WPA3 Personal. Explicați procesul de autentificare pentru WPA3.

Rezolvare: WEP:

- Toate dispozitivele folosesc aceeași cheie de criptare
- Oricine cunoaște cheia poate decripta toate datele
- Vulnerabil la atacuri

WPA3 Personal:

- Fiecare dispozitiv are o cheie de criptare diferită cu AP-ul
- Procesul de autentificare:
  1. Dispozitivul se autentifică cu AP-ul folosind parola rețelei
  2. Dispozitivul și AP-ul fac schimb de chei simetrice de criptare
  3. Comunicația se face cu cheia unică pentru acel dispozitiv

3

Exercițiul 3: Adresare IP (6 puncte)

a) Calculați câte host-uri pot fi în rețeaua 192.168.1.0/24
b) Care este adresa de broadcast pentru această rețea?
c) Care este prima și ultima adresă IP utilizabilă pentru host-uri?

Rezolvare: a) /24 înseamnă 24 biți pentru rețea, 8 biți pentru host

- Numărul de host-uri = 2^8 - 2 = 254 (scădem adresa de rețea și broadcast)

b) Adresa de broadcast = 192.168.1.255

c) Prima adresă utilizabilă: 192.168.1.1
Ultima adresă utilizabilă: 192.168.1.254

4

Exercițiul 4: TCP vs UDP (5 puncte)

Explicați diferențele dintre TCP și UDP. Pentru fiecare din următoarele aplicații, alegeți protocolul potrivit și justificați: streaming video, transfer fișiere, DNS query, joc online.

Rezolvare: TCP:

- Orientat pe conexiune
- Garantează livrarea și ordinea
- Control de flux și congestie
- Overhead mai mare

UDP:

- Fără conexiune
- Nu garantează livrarea
- Mai rapid, overhead mic
- "Fire and forget"

Aplicații:

- **Streaming video**: UDP - viteza e mai importantă decât precizia
- **Transfer fișiere**: TCP - trebuie să garanteze integritatea datelor
- **DNS query**: UDP - query-uri mici, rapid
- **Joc online**: UDP - latența mică e critică

5

Exercițiul 6: DHCP Process (4 puncte)

Descrieți pas cu pas procesul DHCP pentru obținerea unei adrese IP.

Rezolvare:

1. **DHCP_DISCOVER**: Clientul trimite un broadcast pentru a găsi servere DHCP
2. **DHCP_OFFER**: Serverul DHCP răspunde cu o ofertă de adresă IP, mască, gateway, DNS
3. **DHCP_REQUEST**: Clientul confirmă că acceptă oferta printr-un broadcast
4. **DHCP_ACK**: Serverul confirmă alocarea și trimite parametrii finali

6

Exercițiul 7: Frame Analysis (6 puncte)

Analizați următorul frame Ethernet (în hex): `AA BB CC DD EE FF 11 22 33 44 55 66 08 00 45 00 00 1C ...`

Identificați: a) Adresa MAC destinație b) Adresa MAC sursă
c) EtherType d) Ce tip de protocol urmează

Rezolvare: a) **MAC destinație**: AA:BB:CC:DD:EE:FF b) **MAC sursă**: 11:22:33:44:55:66 c) **EtherType**: 08 00 (0x0800) d) **Protocol**: IPv4 (EtherType 0x0800 indică IPv4)

7)

Exercițiul 8: Subnetting (7 puncte)

Împărțiți rețeaua 10.0.0.0/16 în 4 subrețele egale. Calculați pentru fiecare subrețea:

- Adresa de rețea
- Masca de subrețea
- Adresa de broadcast
- Numărul de host-uri disponibile

Rezolvare: Pentru 4 subrețele egale din /16, avem nevoie de 2 biți suplimentari (2^2 = 4). Noua mască: /18 (255.255.192.0)

Subrețeaua 1:

- Rețea: 10.0.0.0/18
- Broadcast: 10.0.63.255
- Host-uri: 2^14 - 2 = 16382

Subrețeaua 2:

- Rețea: 10.0.64.0/18
- Broadcast: 10.0.127.255
- Host-uri: 16382

Subrețeaua 3:

- Rețea: 10.0.128.0/18
- Broadcast: 10.0.191.255
- Host-uri: 16382

Subrețeaua 4:

- Rețea: 10.0.192.0/18
- Broadcast: 10.0.255.255
- Host-uri: 16382

8)

Exercițiul 10: Network Troubleshooting (4 puncte)

Un dispozitiv Raspberry Pi Pico W nu se poate conecta la internet, deși se conectează la Wi-Fi. Enumerați 4 cauze posibile și cum le-ați diagnostica.

Rezolvare:

1. **DHCP nu funcționează**: Verificați dacă dispozitivul a primit adresă IP validă
2. **Gateway incorect**: Testați ping către gateway-ul configurat
3. **DNS nu funcționează**: Încercați conectarea directă la adrese IP în loc de domenii
4. **Firewall/restricții de rețea**: Verificați dacă porturile necesare sunt deschise pe rețea

Curs 9 -> ZigBee

2. ZigBee

Caracteristici principale:

- Standard tehnologic pentru rețele de control și senzori
- Optimizat pentru aplicații IoT cu consum redus și cost mic
- Bazat pe standardul IEEE 802.15.4
- Creat de ZigBee Alliance (Philips, Motorola, HP, Intel)
- Conceput în 1998, standardizat în 2003, revizuit în 2006

Specificații tehnice:

- 65,536 noduri de rețea (client)
- 27 canale pe 2 benzi de frecvență
- 250Kbps viteză de transmisie
- Mesh networking complet
- Optimizat pentru aplicații timing-critical și power management

Frecvențe și performanțe:

- **868.3 MHz** (Europa): 20Kbps, canalul 0
- **902-928 MHz** (Americas): 40Kbps, canale 1-10
- **2405-2480 MHz** (Worldwide): 250Kbps, canale 11-26

Tipuri de dispozitive:

- **ZigBee Coordinator (ZC):** Unul necesar per rețea, inițiază formarea rețelei
- **ZigBee Router (ZR):** Participă la rutarea multi-hop a mesajelor
- **ZigBee End Device (ZED):** Nu permite asociere sau rutare, soluții foarte low-cost

Topologii de rețea:

- **Star:** Dispozitive conectate direct la coordinator
- **Cluster Tree:** Structură arborescentă cu coordinatori și routere
- **Mesh:** Rețea completă cu redundanță și self-healing

Profile publice ZigBee:

- **Home Automation (HA):** Control casnic
- **Smart Energy (SE):** Management energetic
- **Commercial Building Automation (CBA):** Automatizarea clădirilor
- **ZigBee Health Care (ZHC):** Aplicații medicale
- **Telecom Applications (TA):** Aplicații telecom
- **ZigBee RF4CE Remote Control:** Control la distanță

3. LoRa și LoRaWAN

LoRa (Long Range):

- Tehnologie de modulație wireless cu spectru răspândit
- Dezvoltată de Cycleo (2009), achiziționată de SemTech (2012)
- Prima componentă SX1272 lansată în 2013
- Distanță de transmisie mult superioară față de FSK (1-2km)

LoRaWAN:

- Protocol de comunicație optimizat pentru dispozitive cu energie limitată
- Topologie în stea cu stații de bază (BS) și dispozitive finale (ED)
- Stațiile de bază conectate la server central via IP
- Comunicație wireless bazată pe modulația LoRa sau GFSK

Arhitectura rețelei:

- **End Nodes:** Senzori și dispozitive IoT
- **Concentrator/Gateway:** Colectează date de la noduri
- **Network Server:** Procesează și rutează datele
- **Application Server:** Aplicații finale

Clase de dispozitive (EDs):

- **Clasa A (Baseline):** Senzori cu baterie, cea mai eficientă energetic, downlink disponibil doar după TX
- **Clasa B (Baseline):** Actuatori cu baterie, comunicație sincronizată cu beacon, latență controlată
- **Clasa C (Continuous):** Actuatori alimentați de la rețea, pot asculta continuu, fără latență pentru downlink

Adaptive Data Rate (ADR):

- Nodurile apropiate folosesc data rate mare (short range)
- Nodurile îndepărtate folosesc data rate mic (long range)
- Optimizează capacitatea rețelei prin micro-celule locale

4. Sigfox

Caracteristici:

- Implementează propriile stații de bază pentru aplicații IoT
- Distanță de comunicație: kilometri
- Conectează mii de dispozitive
- Viteză: 100 bits per secundă
- Durată de viață: 5 ani

Tehnologii cheie:

- **Repetition**: Fiecare mesaj trimis de 3 ori
  - Time diversity: 3 time slots diferite
  - Frequency diversity: 3 frecvențe diferite
- **Collaborative Network**: 3 stații de bază pentru acoperire (space diversity)
- **Ultra Narrow Band**: Reduce puterea de zgomot și consumul
- 200 mesaje simultane într-un canal de 200kHz

5. NB-IoT (Narrow Band IoT)

Ultra Narrow Band:

- Reduce banda de frecvență a semnalului transmis
- Putere de zgomot redusă
- Permite reducerea puterii de transmisie
- Reduce consumul de energie pentru comunicații radio

Integrare cu LTE:

- **Standalone**: Bandă dedicată de 200kHz
- **Guard band**: În banda de gardă dintre canalele LTE
- **In-band**: Înlocuiește un canal LTE de 200kHz

Curs 10 -> Power Monagement și Surse de Alimentare

① Stabilizatoare de tensiune

-> stabilizatoare liniară : -> utilizează tranzistor/diodă Zener ca element activ, funcționează ca rezistor variabil

-> caracteristici importante -> voltage dropout (diferența minimă $V_{in} - V_{out}$)

-> curentul consumat de stabilizator

-> current limit, output noise

② Familiile 78/79

-> 78XX -> tensiuni pozitive fixe (exp: 7805 = 5V)

-> 79XX -> tensiuni negative

-> LM317 -> tensiune ajustabilă cu formula

$$V_{out} = V_{ref}\left(1 + \frac{R_2}{R_1}\right) + I_{adj} \cdot R_2$$

③ LDO (Low Dropout)

-> avantaje: funcționează cu tensiuni de intrare aproape de ieșire, fără zgomot de comutație

-> dezavantaje: disipă căldură ca stabilizatoarele liniare clasice

④ Surse în comutație

-> Caracteristici : -> reglarea tensiunii prin duty cycle (raport timp pornire / oprire)

-> avantaje : eficiență mai mare, densități de putere mai mare

-> dezavantaje : mai complexe, pot genera EMI, răspuns tranzitii mai lent

-> Tipuri -> Buck (Step-Down) -> coboară tensiunea

-> Boost (Step-up) -> ridică tensiunea

-> Buck-Boost -> poate ridica sau coboară tensiunea

⑤ Surse AC-DC

↳ liniare : transformare + redresare + filtrare + stabilizator liniar
↳ în comutație : mai eficiente, mai compacte pentru puteri mari

⑥ Power Management în Microcontrolere

$P \simeq C \cdot Vdd^2 \times f$ -> tranzistile costă energie

⑦ Starile de Power Management (ATMega328P)

↳ ACTIVE -> toate circuitele funcționale
↳ IDLE -> CPU oprit, periferice active
↳ ADC NOISE REDUCTION : Doar ADC și Timer2 Active
↳ Power-down : cel mai eficient, doar întreruperi externe / watchdog
↳ Power-save : ca power-down + Timer2 asincron
↳ Standby / Extended Standby : Cu oscilator extern

⑧ Consumuri tipice

↳ Active 8 MHz : 5.2 mA
↳ Idle : 1 mA
↳ Power-down : 0.2 - 4 μA (cu / fără watchdog)
↳ Power-save : 0.75 μA

⑨ Circuite de comandă

↳ Releu: → element electromecanic pt comutarea sarcinilor mari
→ necesită driver (tranzistor + diodă flywheel)

⑩ Drivere motoare

↳ punte H → comută polaritatea pt funcționare bidirecțională
↳ implementări: → discret (tranzistori)
→ integrat (L298N)
→ ULN2003 → pt stepper

Exerciții

Exercițiul 1 - Stabilizator Liniar

Enunț: Calculați tensiunea de ieșire pentru un stabilizator LM317 cu R1 = 240Ω și R2 = 1.2kΩ. Vref = 1.25V, Iadj = 50µA.

Rezolvare: Vout = Vref(1 + R2/R1) + Iadj × R2 Vout = 1.25(1 + 1200/240) + 50×10⁻⁶ × 1200 Vout = 1.25(1 + 5) + 0.06 Vout = 1.25 × 6 + 0.06 = 7.5 + 0.06 = **7.56V**

Exercițiul 2 - Eficiența Stabilizatorului

Enunț: Un stabilizator 7805 alimentează o sarcină de 500mA la 5V dintr-o tensiune de 12V. Calculați eficiența și puterea disipată.

Rezolvare: Pout = Vout × Iout = 5V × 0.5A = 2.5W Pin = Vin × Iin = 12V × 0.5A = 6W (presupunem Iin ≈ Iout) Eficiența η = Pout/Pin = 2.5/6 = **41.7%** Pdisipat = Pin - Pout = 6 - 2.5 = **3.5W**

Exercițiul 3 - Calcul Consum Power Management

Enunț: Un microcontroller ATMega328P (3.3V) funcționează astfel:

- 100ms activ la 8MHz (5.2mA)
- 900ms în power-save (0.75µA) Calculați consumul mediu și durata de viață cu o baterie de 200mAh.

Rezolvare: Imediu = (Iactiv × tactiv + Ipsave × tpsave) / ttotal Imediu = (5.2mA × 100ms + 0.75µA × 900ms) / 1000ms Imediu = (0.52mA×ms + 0.000675mA×ms) / 1000ms Imediu = 0.520675mA×ms / 1000ms = **0.521mA**

Durata = Capacitate/Imediu = 200mAh/0.521mA = **384 ore ≈ 16 zile**

Exercițiul 4 - Duty Cycle în Surse Comutate

Enunț: O sursă buck trebuie să transforme 12V în 5V. Calculați duty cycle-ul teoretic necesar.

Rezolvare: Pentru sursa buck: Vout = Vin × D Unde D este duty cycle-ul D = Vout/Vin = 5V/12V = **0.417 = 41.7%**

Exercițiul 5 - Dimensionarea Rezistorului pentru Releu

Enunț: Un releu cu bobina de 12V/50mA trebuie comandat de un pin de 5V prin tranzistorul BC109 (hFE = 200). Calculați rezistența de bază necesară.

Rezolvare: Ib = Ic/hFE = 50mA/200 = 0.25mA Vbe ≈ 0.7V pentru Si R = (Vin - Vbe)/Ib = (5V - 0.7V)/0.25mA = 4.3V/0.25mA = **17.2kΩ**

Valoare standard apropiată: **18kΩ**

Exercițiul 6 - Optimizare Frecvență ADC

Enunț: Un ADC cu prescaler 128 la 8MHz necesită 13 cicluri pentru conversie. Calculați timpul de conversie și frecvența ADC rezultată.

Rezolvare: fADC = fCPU/prescaler = 8MHz/128 = 62.5kHz tconversie = 13 cicluri / fADC = 13/62500 = **208µs** Frecvența ADC: **62.5kHz** (în intervalul optim 50-200kHz)

Exercițiul 7 - Comparație Moduri Power Management

Enunț: Comparați consumul unui sistem care rulează 1ms din fiecare secundă în: a) Mod normal (5mA activ) b) Idle între operații (1mA idle) c) Power-save între operații (1µA)

Rezolvare: a) Imediu_normal = 5mA (continuu)

b) Imediu_idle = (5mA × 1ms + 1mA × 999ms)/1000ms = (5 + 999)/1000 = **1.004mA**

c) Imediu_psave = (5mA × 1ms + 0.001mA × 999ms)/1000ms = (5 + 0.999)/1000 = **≈6µA**

Concluzie: Power-save oferă o reducere de ~167 ori față de idle.